

PROJETO: INTEGRA+



PUC
CAMPINAS
PONTIFÍCIA UNIVERSIDADE CATÓLICA

Curso: Engenharia da Computação

Disciplina: PI: Projeto e Implementação de Aplicativos **Código:** 13132

Turma: 101 ou 102

Professor: Fernando Luiz de Almeida Silveira

Nome do Grupo: INTEGRA+

1. Felipe Natali Amaral – RA:24792566
2. José Ricardo Padovan Ferreira – RA: 24010692
3. Pedro Pimentel Rodrigues– RA: 24023362
4. Pedro Guimarães Peruchi – RA: 24019888
5. Tiago Aureliano Lança Rodriguez – RA: 25004196

Campinas, 1º semestre de 2026

1.Implementação do Front-End

O front-end do sistema foi desenvolvido com base no protótipo elaborado na segunda entrega. Foram utilizadas as tecnologias HTML, CSS e JavaScript para a construção das interfaces.

A navegação entre as páginas segue o modelo definido na etapa de prototipação, garantindo coerência com a proposta inicial do projeto.

O repositório do front-end foi organizado em pastas separadas, contendo *pages*, *scripts* e *styles*, de forma a facilitar a manutenção e a escalabilidade do sistema. Além disso, foram definidas variáveis de estilo para padronização visual da aplicação.

Atualmente, o front-end encontra-se funcional e integrado ao back-end, realizando consumo de dados provenientes da API e do banco de dados.

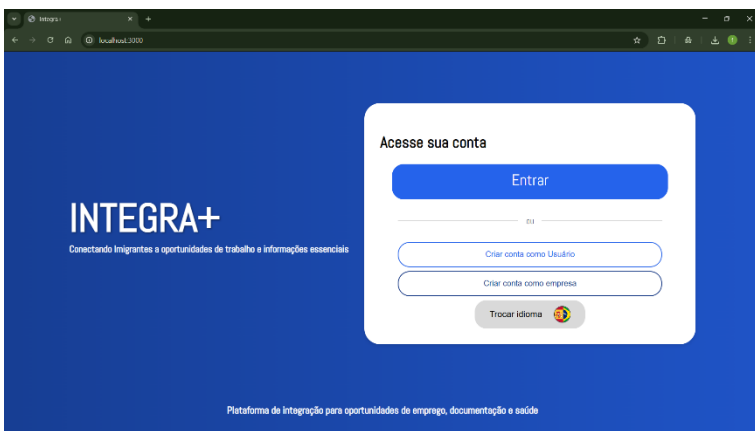


Figura 1:Primeira tela(index)

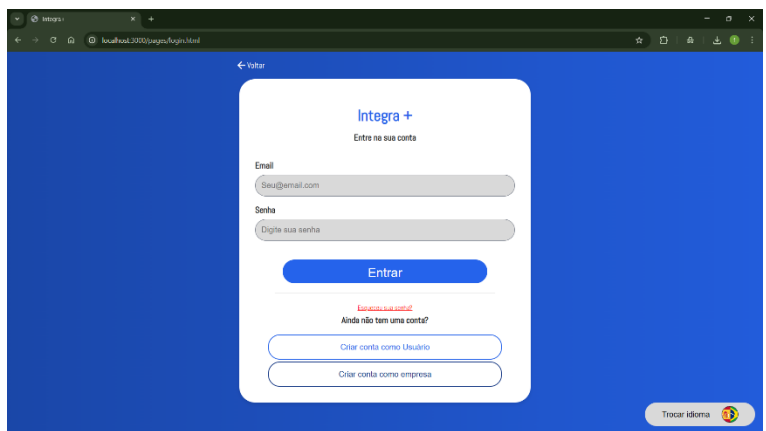


Figura 2:Tela de Login

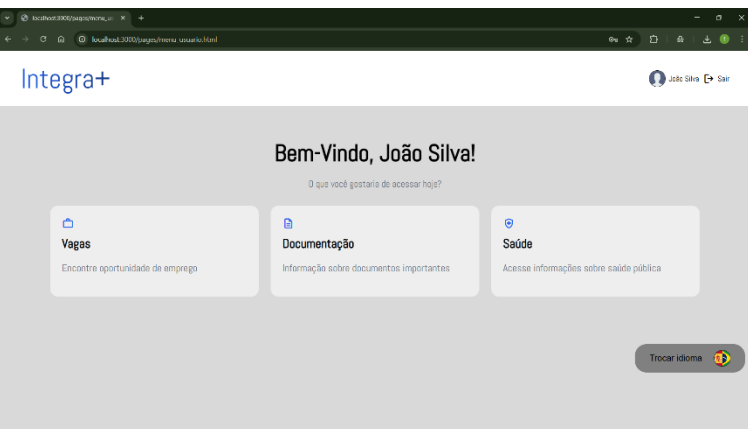


Figura 3:Menu Principal

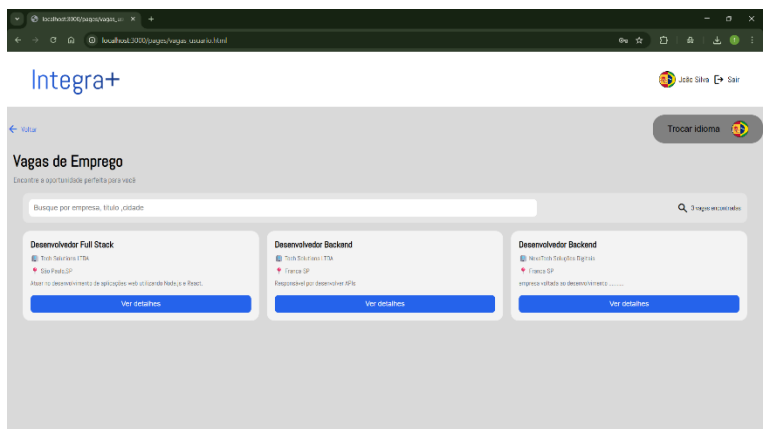


Figura 3:Tela de vagas

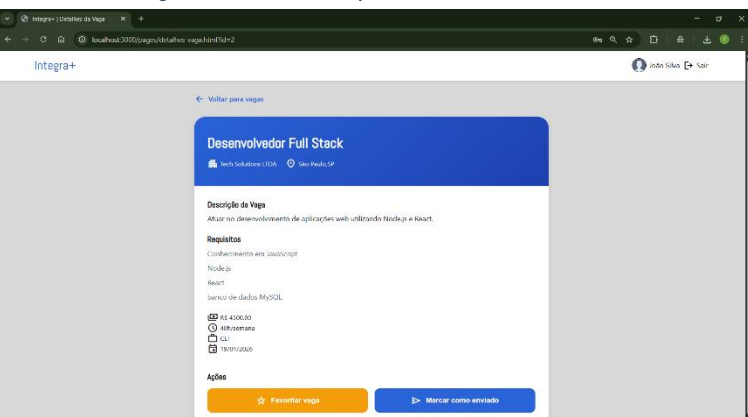


Figura 4: tela de vaga(já consumindo dados do banco

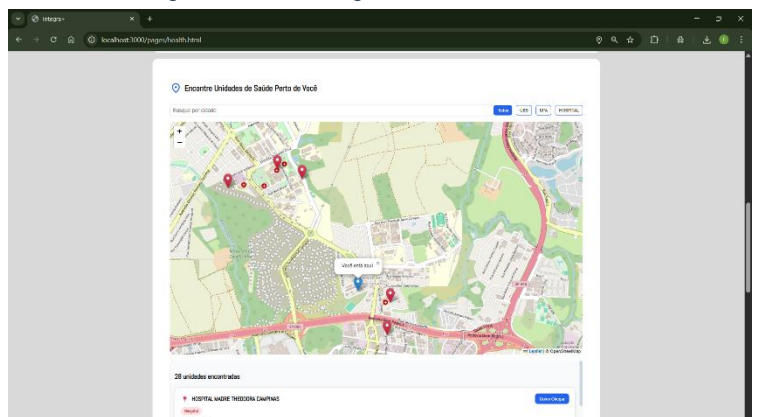


Figura 5:Tela de saúde consumindo dados do

O sistema segue o mesmo fluxo definido na etapa de prototipação, sendo separado em blocos. O primeiro corresponde à área de login, que contempla cadastro de usuário e empresa, além da recuperação de senha.

Após o login, o fluxo se divide entre os perfis de usuário e empresa.

No menu do usuário, é possível visualizar as vagas disponíveis, favoritar oportunidades e acessar informações relacionadas à documentação e saúde.

Já no menu da empresa, é possível visualizar todas as vagas cadastradas no sistema, além de cadastrar e editar suas próprias vagas.

Organização:

O front-end do sistema foi estruturado de forma organizada, separando os arquivos em diretórios distintos para facilitar a manutenção e escalabilidade do projeto.

Foram criadas três principais divisões: *HTML*, *scripts* (JavaScript) e *styles* (CSS), permitindo a separação clara entre estrutura, lógica e estilo da aplicação.

A seguir, são apresentados registros visuais da organização dos arquivos no repositório do projeto.



Figura 6: telas do sistema (Html)

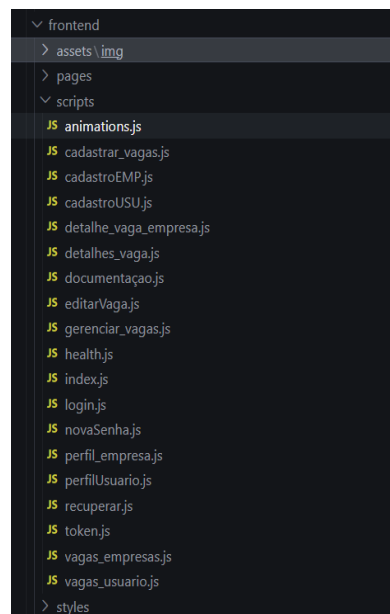


Figura 7: scripts

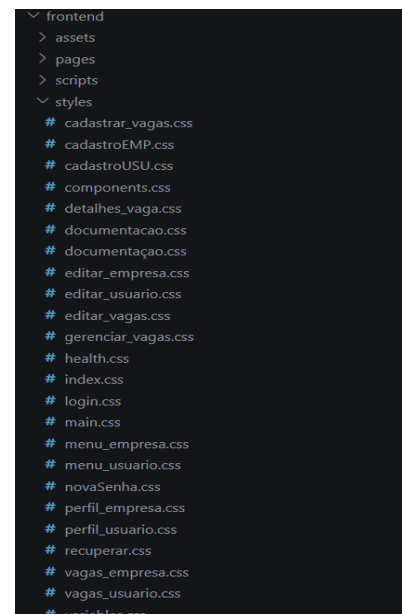


Figura 8: styles

2. Back-End em funcionamento

O back-end foi desenvolvido seguindo uma arquitetura em camadas, composta por *controllers*, *models*, *routes*, *config* e *app/server*. Essa estrutura permite a separação de responsabilidades, facilitando a manutenção e a evolução do sistema.

O sistema conta com mais de 20 rotas implementadas e testadas, utilizando requisições dos tipos GET, POST e DELETE. Essas rotas são responsáveis por operações como consulta, cadastro e remoção de dados, garantindo o funcionamento das principais funcionalidades da aplicação.

A conexão com o banco de dados foi realizada de forma local, com o objetivo de desenvolver e validar o funcionamento do sistema durante o processo de implementação.

Fluxo da API:

As requisições no sistema seguem um fluxo estruturado, onde a rota recebe a requisição do front e a direciona para o controller correspondente. O controller processa a lógica da aplicação e, interage com o model para comunicação com o banco de dados. Após a execução, a resposta é retornada ao cliente em formato JSON.

Conexão:

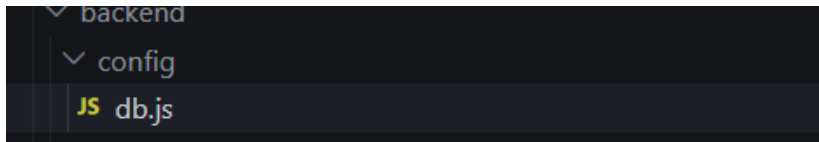


Figura 9: Arquivo de conexão

A conexão com o banco de dados foi realizada utilizando o método `createConnection`, de forma local, conectando-se ao banco *integraplus*. Essa conexão permite a comunicação entre o back-end e o banco de dados para execução das operações do sistema.

Exemplo de rotas:

Autenticação:

`/api/login`
`/api/cadastro`
`/api/usuario`

Manipulação das vagas:

`/api/vagas`
`/api/vagaById`
`/api/vaga(POST)`
`/api/vaga (DELETE)`

Unidades:

`/api/unidades`

Testes realizados:

Os testes das rotas foram realizados utilizando a ferramenta Postman, validando o funcionamento das requisições da API.

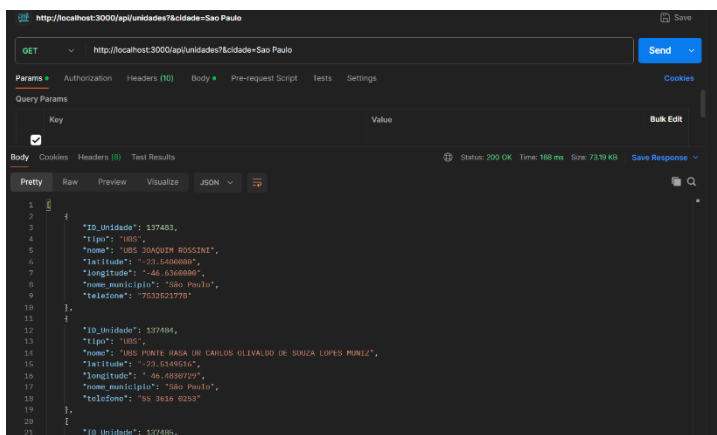


Figura 10: Teste de consulta de dados via requisição GET

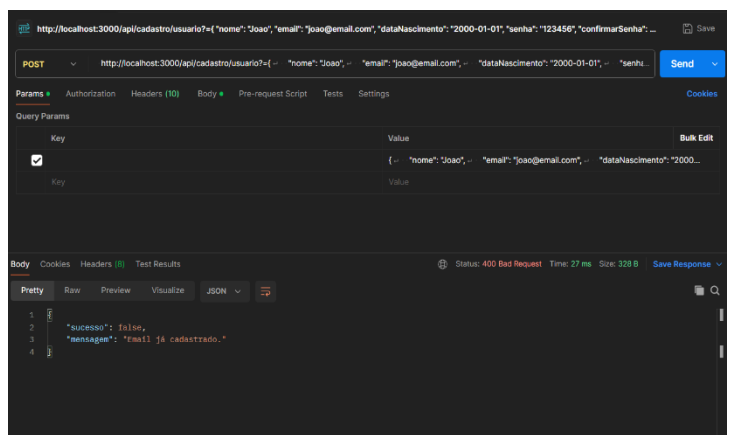


Figura 11: Teste de cadastro de usuário já cadastrado

3. Banco de Dados Finalizado

O modelo entidade-relacionamento (MER/DER) foi atualizado com o objetivo de melhorar a funcionalidade da aplicação e adequar o banco de dados às necessidades do sistema. Como exemplo de alteração, foi adicionada a informação de *cidade* como atributo da entidade vaga, permitindo maior detalhamento das oportunidades cadastradas.

Essas alterações contribuíram para a melhoria da organização dos dados e para uma melhor representação das regras de negócio do sistema.

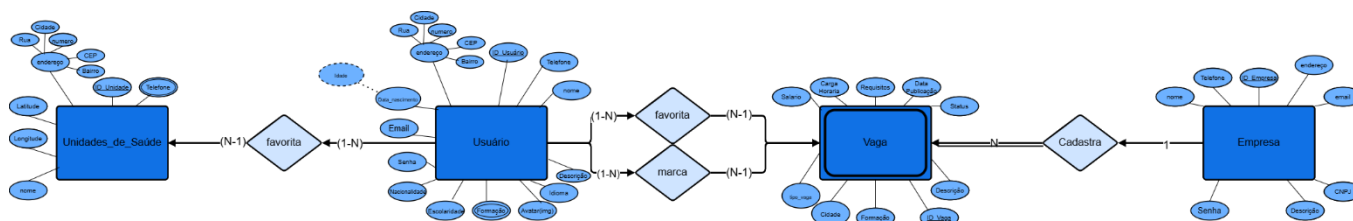


Figura 13: MER atualizado

Link no modelo:

https://app.diagrams.net/#G1AOnnXGchNkgOAlpapp_6oxo3ntmBJNi#%7B%22pageId%22%3A%22sUmt_y0VNR6pviyInWbH%22%7D

Scripts SQL:

Para a construção do banco de dados foi utilizado o SGBD MySQL, no qual foram criadas as tabelas necessárias para o funcionamento do sistema. Essas tabelas foram estruturadas com base no modelo entidade-relacionamento (MER), definindo atributos, tipos de dados e restrições como chaves primárias e únicas.

```
CREATE TABLE `usuario` (
  `ID_usuario` int NOT NULL AUTO_INCREMENT,
  `nome` varchar(100) NOT NULL,
  `email` varchar(100) NOT NULL,
  `telefone` varchar(15) DEFAULT NULL,
  `data_nascimento` date DEFAULT NULL,
  `senha` varchar(255) NOT NULL,
  `endereco` varchar(150) DEFAULT NULL,
  `idioma` varchar(50) DEFAULT NULL,
  `nacionalidade` varchar(50) DEFAULT NULL,
  `escolaridade` varchar(50) DEFAULT NULL,
  `formacao` varchar(100) DEFAULT NULL,
  `avatar` varchar(255) DEFAULT NULL,
  `descricao` varchar(500) DEFAULT NULL,
  PRIMARY KEY (`ID_usuario`),
  UNIQUE KEY `email` (`email`))
```

Figura 14: Tabela Usuário

```
CREATE TABLE `empresa` (
  `ID_empresa` int NOT NULL AUTO_INCREMENT,
  `nome` varchar(100) NOT NULL,
  `email` varchar(100) NOT NULL,
  `telefone` varchar(15) DEFAULT NULL,
  `endereco` varchar(150) DEFAULT NULL,
  `senha` varchar(255) NOT NULL,
  `cnpj` varchar(20) DEFAULT NULL,
  `descricao` varchar(500) DEFAULT NULL,
  PRIMARY KEY (`ID_empresa`),
  UNIQUE KEY `email` (`email`),
  UNIQUE KEY `cnpj` (`cnpj`))
```

Figura 15: Tabela Empresa

```
CREATE TABLE `unidade_de_saude` (
  `ID_Unidade` int NOT NULL AUTO_INCREMENT,
  `nome` varchar(200) DEFAULT NULL,
  `endereco` varchar(150) DEFAULT NULL,
  `telefone` varchar(50) DEFAULT NULL,
  `latitude` decimal(10,7) DEFAULT NULL,
  `longitude` decimal(10,7) DEFAULT NULL,
  `tipo` varchar(100) DEFAULT NULL,
  `nome_municipio` varchar(100) DEFAULT NULL,
  PRIMARY KEY (`ID_Unidade`))
```

Figura 16: tabela Unidades Saúde

```
CREATE TABLE `vaga` (
  `ID_vaga` int NOT NULL AUTO_INCREMENT,
  `ID_empresa` int NOT NULL,
  `titulo` varchar(100) NOT NULL,
  `descricao` varchar(500) NOT NULL,
  `salario` decimal(10,2) DEFAULT NULL,
  `data_publicacao` datetime DEFAULT CURRENT_TIMESTAMP,
  `carga_horaria` int DEFAULT NULL,
  `requisitos` varchar(500) NOT NULL,
  `tipo_vaga` varchar(20) DEFAULT NULL,
  `status` varchar(20) DEFAULT NULL,
  `cidade` varchar(100) DEFAULT NULL,
  PRIMARY KEY (`ID_vaga`),
  KEY `FK_empresa_vaga` (`ID_empresa`),
  CONSTRAINT `FK_empresa_vaga` FOREIGN KEY (`ID_empresa`) REFERENCES `empresa` (`ID_empresa`),
  CONSTRAINT `vaga_chk_1` CHECK ((`salario` > 0))
```

Figura 17: Tabela Vaga

Relacionamentos N:N

```
CREATE TABLE `usuario_favorita_vaga` (
  `ID_usuario` int NOT NULL,
  `ID_vaga` int NOT NULL,
  PRIMARY KEY (`ID_usuario`, `ID_vaga`),
  KEY `fk_favorita_vaga` (`ID_vaga`),
  CONSTRAINT `fk_favorita_vaga` FOREIGN KEY (`ID_vaga`) REFERENCES `vaga` (`ID_vaga`) ON DELETE CASCADE,
  CONSTRAINT `usuario_favorita_vaga_ibfk_1` FOREIGN KEY (`ID_usuario`) REFERENCES `usuario` (`ID_usuario`))
```

Figura 18: Relacionamento usuário favorita vaga

```
CREATE TABLE `usuario_favorita_unidade` (
  `ID_usuario` int NOT NULL,
  `ID_Unidade` int NOT NULL,
  PRIMARY KEY (`ID_usuario`, `ID_Unidade`),
  KEY `ID_Unidade` (`ID_Unidade`),
  CONSTRAINT `usuario_favorita_unidade_ibfk_1` FOREIGN KEY (`ID_usuario`) REFERENCES `usuario` (`ID_usuario`),
  CONSTRAINT `usuario_favorita_unidade_ibfk_2` FOREIGN KEY (`ID_Unidade`) REFERENCES `unidade_de_saude` (`ID_Unidade`))
```

Figura 19:usuário favorita unidade de saúde

```
CREATE TABLE `usuario_marca_vaga` (
  `ID_usuario` int NOT NULL,
  `ID_vaga` int NOT NULL,
  PRIMARY KEY (`ID_usuario`, `ID_vaga`),
  KEY `fk_marca_vaga` (`ID_vaga`),
  CONSTRAINT `fk_marca_vaga` FOREIGN KEY (`ID_vaga`) REFERENCES `vaga` (`ID_vaga`) ON DELETE CASCADE,
  CONSTRAINT `usuario_marca_vaga_ibfk_1` FOREIGN KEY (`ID_usuario`) REFERENCES `usuario` (`ID_usuario`))
```

Figura 20:usuário marca vaga

O banco de dados também foi populado com aproximadamente 68 mil registros de unidades de saúde, obtidos por meio de uma base de dados pública. Essa inserção teve como objetivo enriquecer o sistema com dados reais, permitindo testes de busca, filtragem e consulta em maior escala.

Conexão com o back-end:

A integração entre o banco de dados e o back-end é realizada por meio da arquitetura em camadas do sistema. Os controllers recebem as requisições da API e acionam os models, responsáveis por executar as operações de consulta, inserção, atualização e remoção de dados no banco. Dessa forma, o fluxo garante a comunicação entre a aplicação e o banco de dados de forma organizada e estruturada.

Exemplo de consulta:

```
const getVaga = async (ids) => {
  try {
    const placeholders = ids.map(() => '?').join(',');
    const [rows] = await db.promise().query(
      `SELECT v.*, e.nome AS empresa
      FROM vaga v
      JOIN empresa e ON v.ID_empresa = e.ID_empresa
      WHERE ID_vaga IN (${placeholders})`,
      ids
    );
    return rows;
  } catch (error) {
    console.error("Erro ao obter vaga:", error);
    throw error;
  }
};
```

Figura 21: Exemplo de consulta ao banco

4. Integração das Camadas

O sistema já está integrado entre os três principais ambientes: front-end, back-end e banco de dados. Essa integração permite o fluxo completo de dados dentro da aplicação, desde a interação do usuário na interface até o armazenamento e recuperação das informações no banco de dados.

A seguir é apresentado um exemplo de implementação que demonstra essa comunicação entre as camadas do sistema:

```
try {
  const response = await fetch("http://localhost:3000/api/cadastro/usuario", {
    method: "POST",
    headers: {
      "Content-Type": "application/json"
    },
    body: JSON.stringify({
      nome,
      email,
      dataNascimento,
      senha,
      confirmarSenha
    })
  });
};
```

Figura 22: requisição do front

O front-end realiza a captura dos dados inseridos pelo usuário e os envia para a API do back-end por meio de requisições HTTP. No back-end, o controller recebe essas informações, processa a requisição e aciona o model, que realiza as operações necessárias no banco de dados, como inserção ou consulta de dados.

```
const cadastrarUsuario = async (req, res) => {
  try {
    const {
      nome,
      email,
      dataNascimento,
      senha,
      confirmarSenha
    } = req.body;

    if (!nome || !email || !dataNascimento || !senha || !confirmarSenha) {
      return res.status(400).json({
        sucesso: false,
        mensagem: 'Todos os campos do cadastro de usuário são obrigatórios.'
      });
    }

    if (senha !== confirmarSenha) {
      return res.status(400).json({
        sucesso: false,
        mensagem: 'A confirmação de senha não confere.'
      });
    }
  }
};
```

Figura 23: Controller recebe os dados

```
const existe = await cadastroModel.verificaExistencia(email);

if (existe.length > 0) {
  return res.status(400).json({
    sucesso: false,
    mensagem: 'Email já cadastrado.'
  });
}

const senhaHash = await bcrypt.hash(senha, 10);
await cadastroModel.criarUsuario(nome, email, dataNascimento, senhaHash);

return res.status(201).json({
  sucesso: true,
  mensagem: 'Usuário cadastrado com sucesso.'
});

} catch (error) {
  console.error(error);
  return res.status(500).json({
    sucesso: false,
    mensagem: 'Erro ao cadastrar usuário.',
    erro: error.message
  });
};
```

Figura 24: Criptografia da senha e envio dos dados ao model

```
const criarUsuario = async (nome, email, dataNascimento, senha) => {  
  const [result] = await db.promise().query(  
    `INSERT INTO usuario  
      (nome, email, data_nascimento, senha)  
    VALUES (?, ?, ?, ?)`,  
    [nome, email, dataNascimento, senha]  
  );  
  return result;  
};
```

Figura 25: Model insere os dados no banco

5. Evolução em relação à 2ª entrega

Em relação à segunda entrega, o projeto evoluiu significativamente com a implementação completa do front-end, no qual foram desenvolvidas todas as telas do sistema. O back-end também foi ampliado, com a implementação de todas as rotas, controllers e models necessários para o funcionamento da aplicação. Além disso, foi realizada a integração entre as camadas do sistema e o banco de dados foi finalizado.

Durante o desenvolvimento, foram identificados alguns desafios. O principal deles foi a integração de códigos desenvolvidos por diferentes integrantes do grupo, o que exigiu padronização das lógicas e da estrutura do sistema. Outro problema encontrado foi relacionado à utilização de tokens para segurança dos dados, sendo necessária a implementação de middlewares para validação e decodificação das informações.

Um problema ainda em aberto está relacionado aos dados de latitude provenientes da base de consulta pública, que estão restritos a valores entre 0 e 10 graus na região norte do país. Para solucionar essa questão, será necessário realizar um novo filtro ou reprocessamento da base de dados utilizada.

O que ainda falta para a entrega final:

Para a entrega final, ainda será necessário implementar o sistema de recuperação de senha, além da funcionalidade de edição de perfis de usuário e empresa. Também será realizada uma melhoria na padronização da estrutura do código, visando maior organização e manutenção do sistema.

Além disso, está prevista a implementação de um sistema de tradução de páginas, com o objetivo de melhorar a acessibilidade e a experiência do usuário.